HealthMate

Mobile App Project Blueprint

Planning version 2.0 | 6 November 2026

Coursework: CI6330 - Mobile Application Development, Coursework 1

Repository: github.com/navod718/healthmate

Account restriction: Use only Navod718 for project GitHub work.

1. Goal and scope

Build an offline mobile app for recording daily steps, calories burned and water intake, with a clear dashboard and searchable history. Develop a native Android application using Kotlin, Jetpack Compose and Room. The submission build is an Android release APK.

Technology clarification: This revision follows the user-reported lecturer instruction to use Kotlin, Jetpack Compose and Room, with no Flutter. The original draft specifies sqflite; Room replaces it in this plan. Retain the lecturer clarification with the project records.

2. App screens

Screen	Main functions
Dashboard	Today's date, total steps, calories and water; quick access to add a record.
Records	Records newest first, date filtering and access to edit or delete.
Add / Edit Record	Date selection, health inputs, validation and save.
Delete confirmation	Confirm before permanently removing a record.

Navigation: Dashboard and Records tabs, with a prominent Add Record button.

3. Proposed record rules

• Allow multiple entries per day; sum them for the dashboard.

• Store dates as YYYY-MM-DD and accept non-negative whole numbers.

• Require every field, with at least one health value greater than zero.

• Prevent future dates and preserve records across app restarts.

• Provide loading, empty, success and error states.

These record rules are proposed design choices. The brief does not prescribe the number of entries per day or prohibit future dates.

4. Technology and architecture

Component	Choice	Purpose
Language	Kotlin	Native Android application development
UI	Jetpack Compose / Material 3	Declarative screens and Android UI components
State	ViewModel / StateFlow	Lifecycle-aware screen state and actions
Database	Room / SQLite	Local persistence using entities and DAOs
Async work	Kotlin coroutines / Flow	Database work and reactive record updates
Dates	java.time	Date validation, storage and display

Use MVVM with a repository layer, organized by feature. Screens present the UI; viewmodels manage state and actions; the repository performs database operations.

Data flow: Compose screen → ViewModel → Repository → Room DAO → SQLite. Flow updates are exposed as StateFlow and collected by the UI with lifecycle awareness.

```
com.navod718.healthmate/
  MainActivity.kt
  app/                  # App entry and navigation
  core/database/        # RoomDatabase configuration
  core/validation/
  feature/health/
    data/local/         # Room entity and DAO
    data/repository/
    presentation/       # ViewModels, screens, components
  ui/theme/
```

5. Database design

Required table: health_records

Field	SQLite type	Rules
id	INTEGER	Primary key, auto-increment
date	TEXT	Required; YYYY-MM-DD
steps	INTEGER	Required; non-negative
calories	INTEGER	Required; non-negative
water	INTEGER	Required; non-negative, in ml

Map a Room @Entity to health_records, with an auto-generated @PrimaryKey. Use DAO @Insert, @Update and @Delete methods plus parameterized @Query methods for date filtering and SUM totals. Return Flow for reactive reads and use suspend functions for writes. Provide an explicit Load demo data action, clearly identifying sample records and preventing accidental duplicate seeding.

6. Visual direction

• Light background, high-contrast text and readable typography.

• Steps: green with a walking icon. Calories: orange/red with a flame icon. Water: blue with a water-drop icon.

• Summary cards, clearly labelled inputs and large touch targets.

• Responsive layouts that accommodate small phones and larger text settings.

7. Development milestones

Stage	Result
1. Design	Wireframes, architecture diagram and database diagram
2. Foundation	Android Studio project, Compose theme, navigation and ViewModel/StateFlow setup
3. Storage	Room entity, DAO, database, repository, CRUD and demo records
4. Features	Add/edit form, records list, date filter and dashboard
5. Verification	Automated checks and device tests with evidence
6. Submission	Release APK, report, narrated video and accessible links

Commit each meaningful completed step to navod718/healthmate. Preserve genuine progressive development history and describe changes clearly in commit messages.

8. Testing plan

ID	Scenario and expected outcome
T01	Add a valid record; it appears in history and contributes to totals.
T02	Missing, negative or non-numeric input is rejected with a useful message.
T03	Edit a record; saved values and dashboard totals update.
T04	Cancel deletion to preserve a record; confirm deletion to remove it.
T05	Filter by date; show only matching records or an empty state.
T06	Add multiple entries for today; dashboard sums them correctly.
T07	Restart the app; saved data remains available.
T08	Verify empty states and explicit loading of demo records.

Run the tests before recording Actual Result and Status. Each test record must include Test ID, Scenario, Input, Expected Result, Actual Result and Status, with evidence where appropriate.

9. Report and submission checklist

• Professional PDF report with cover page and table of contents.

• Project overview and features; architecture explanation, diagram, folders and state management.

• Database design, field descriptions, entity diagram and selected database code.

• CRUD explanations with selected code snippets; packages and reasons for selection.

• Wireframes, real app screenshots and UI/UX design decisions.

• At least 3 meaningful tests; target 5-10 tests with evidence.

• Significant technical issues and fixes; repository link and progressive Git history.

• Conclusion and Harvard references.

Report formatting: Times New Roman; main headings 16 pt, subheadings 14 pt and body 11 pt. Include page numbers, captions and readable screenshots.

Appendix	Required link
A	Demonstration video: your own narration, maximum 25 MB, Google Drive sharing set to Anyone with the link - Viewer.
B	Release APK: Google Drive sharing set to Anyone with the link - Viewer.
C	Complete source code in the public Navod718/healthmate repository.
D (optional)	Additional screenshots, tests, database evidence and Git evidence.

10. Assessment priorities

Area	Marks
Application architecture	20
UI/UX design	10
Database integration	30
Documentation and testing	25
Deliverables	15
Total	100

Stated deadline: 23 November 2026 at 3:55 pm. The brief does not specify a timezone; confirm through ELMS.

11. Planning notes

Complete the core scope before adding goals or charts. The brief is marked Draft - Subject to Moderation and contains conflicting individual/group submission wording; confirm the final instructions with the lecturer or ELMS.

The brief requires your own core writing, creativity, analysis and reasoning, plus an Acknowledgement of Generative AI (GAI) Contribution describing actual AI use. This blueprint is an AI-assisted planning document, not a completed technical report or evidence of implementation.

Source: User-provided CW1.docx, Kingston University Coursework Brief 2026/2027, CI6330, Coursework 1 - HealthMate. Technical choices and proposed behavior reflect the project blueprint discussed in chat.